

Theory of Computation

Introduction

Samit Biswas¹

¹Department of Computer Science and Technology,
Indian Institute of Engineering Science and Technology, Shibpur
Email: samit@cs.iiests.ac.in

Table of Contents

- 1 Introduction
- 2 Mathematical Preliminaries
- 3 Formal Language
- 4 Schedule of Lectures

- **Theory of Computation** studies the solvability and efficiency of problems using formal models of computation in computer science and mathematics.

- **Theory of Computation** studies the solvability and efficiency of problems using formal models of computation in computer science and mathematics.
- The field is generally divided into three central pillars:
 - Automata Theory,
 - Computability Theory, and
 - Complexity Theory.

- **Automata Theory:**
 - **Automata Theory:** studies abstract machines and the languages they recognize.

- **Automata Theory:**

- **Automata Theory:** studies abstract machines and the languages they recognize.

- **Computability Theory:**

- **Computability Theory** studies which problems are solvable by computers and which are not, irrespective of **time** or **memory** constraints.

- **Automata Theory:**

- **Automata Theory:** studies abstract machines and the languages they recognize.

- **Computability Theory:**

- **Computability Theory** studies which problems are solvable by computers and which are not, irrespective of **time** or **memory** constraints.

- **Complexity Theory:**

- **Complexity theory** shifts the focus from "Can it be solved?" to "How many resources does it take?" The primary resources are **Time** (CPU cycles) and **Space** (Memory).

Some Basics:

- **Automaton** = A self-operating machine or mechanism (Dictionary definition), plural is **Automata**.

Some Basics:

- **Automaton** = A self-operating machine or mechanism (Dictionary definition), plural is **Automata**.
- **Automata** = abstract computing devices.

Some Basics:

- **Automaton** = A self-operating machine or mechanism (Dictionary definition), plural is **Automata**.
- **Automata** = abstract computing devices.
- **Automata theory** = the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems, and the computational problems that can be solved using these machines.

Some Basics:

- **Automaton** = A self-operating machine or mechanism (Dictionary definition), plural is **Automata**.
- **Automata** = abstract computing devices.
- **Automata theory** = the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems, and the computational problems that can be solved using these machines.
 - Mathematical models of computation
 - Finite automata
 - Push-down automata
 - Turing machines

History

- **1930s** : Alan Turing defined machines more powerful than any in existence, or even any that we could imagine – Goal was to establish the boundary between what was and was not computable.

History

- **1930s** : Alan Turing defined machines more powerful than any in existence, or even any that we could imagine – Goal was to establish the boundary between what was and was not computable.
- **1940s** : In an attempt to model “Brain function” researchers defined finite state machines.

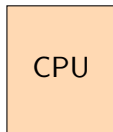
History

- **1930s** : Alan Turing defined machines more powerful than any in existence, or even any that we could imagine – Goal was to establish the boundary between what was and was not computable.
- **1940s** : In an attempt to model “Brain function” researchers defined finite state machines.
- **Late 1950s** : Linguist Noam Chomsky began the study of Formal Grammars.

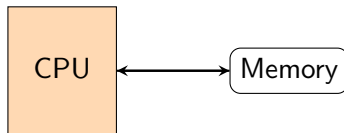
History

- **1930s** : Alan Turing defined machines more powerful than any in existence, or even any that we could imagine – Goal was to establish the boundary between what was and was not computable.
- **1940s** : In an attempt to model “Brain function” researchers defined finite state machines.
- **Late 1950s** : Linguist Noam Chomsky began the study of Formal Grammars.
- **1960s** : A convergence of all this into a formal theory of computer science, with very deep philosophical implications as well as **practical applications** (compilers, web searching, hardware, A.I., algorithm design, software engineering, . . .)

A widely accepted model of computation:

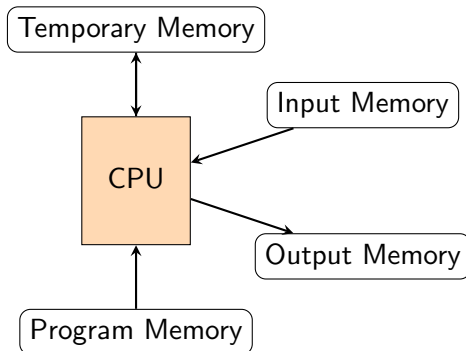


A widely accepted model of computation:



Computation

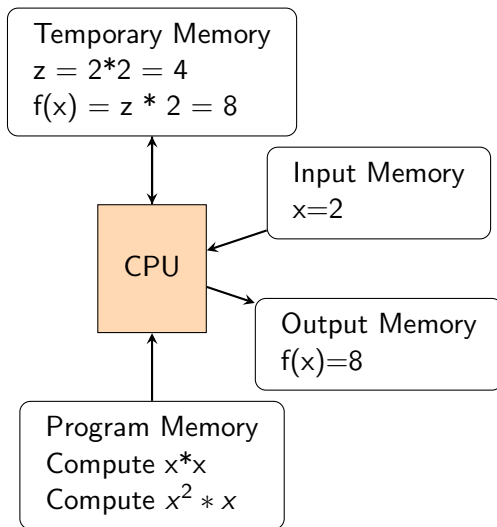
The different Components:



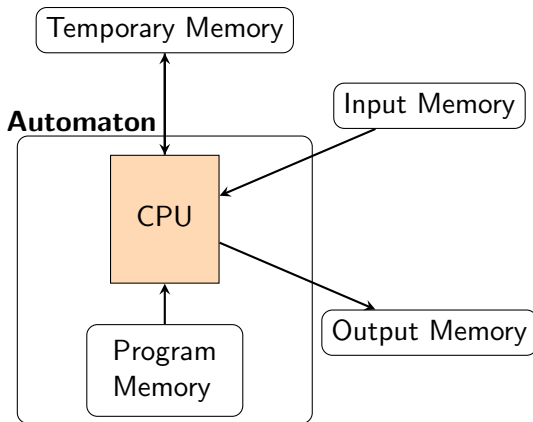
Example: $f(x) = x^3$

Computation

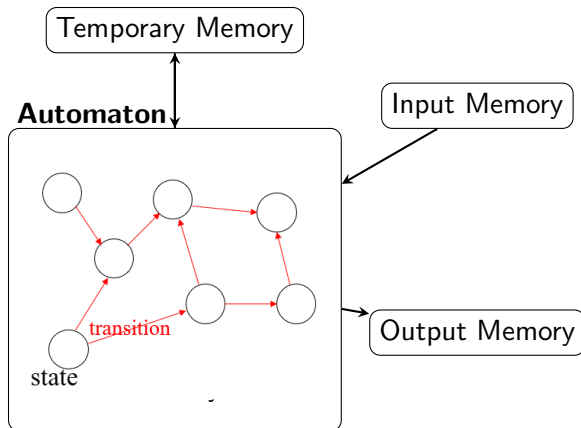
Example: $f(x) = x^3$



Automaton



Automaton

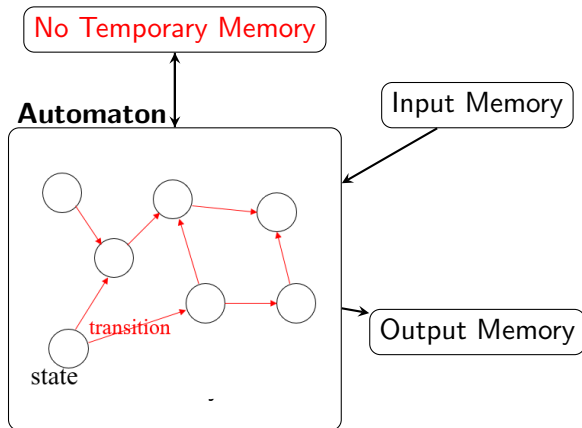


Different Kinds of Automata

Automata are distinguished by the temporary memory

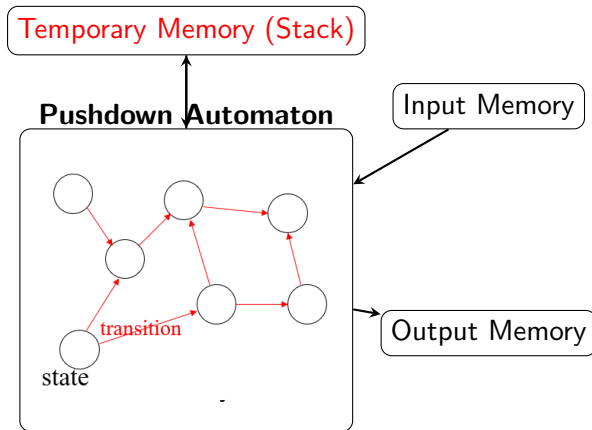
- **Finite Automata:** no temporary memory
- **Pushdown Automata:** stack
- **Turing Machines:** random access memory

Finite Automaton



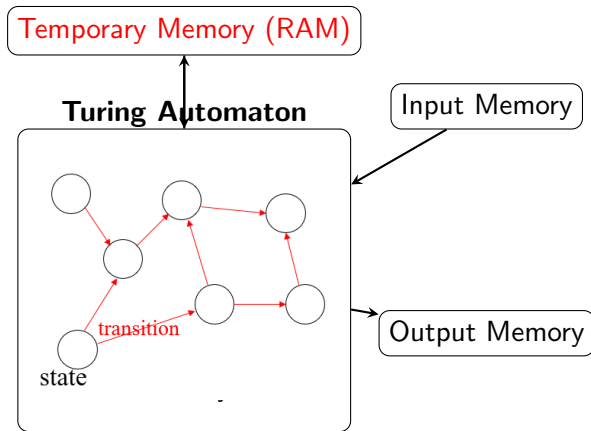
Example: Elevators, Vending Machines
(small computing power)

Pushdown Automaton



Example: Compilers for Programming Languages
(medium computing power)

Turing Machine



Example: Any Algorithm (highest computing power)

Automata

Simple problems

More complex problems

Hardest problems

Finite Automata



Pushdown Automata



Turing Machine

Less Power



More Power

Solve more
computational problems

Time Complexity of Computational Problems:

- **NP-complete problems:** Believed to take exponential time to be solved
- **P problems:** Solved in polynomial time

Table of Contents

- 1 Introduction
- 2 Mathematical Preliminaries**
- 3 Formal Language
- 4 Schedule of Lectures

Sets: A set is a collection of elements.

- $A = \{1, 2, 4\}$

$$B = \{train, bus, bicycle, airplane\}$$

Mathematical Preliminaries

Sets: A set is a collection of elements.

- $A = \{1, 2, 4\}$
 $B = \{train, bus, bicycle, airplane\}$
- we write:
 $1 \in A$
 $ship \notin B$

Mathematical Preliminaries

Set Representations

- $C = \{a, b, c, d, e, f, g, h, i, j, k\}$

$C = \{a, b, \dots, k\}$ → Finite Set

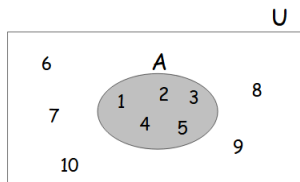
$S = \{2, 4, 6, \dots\}$ → Infinite Set

$S = \{j : j > 0, \text{ and } j = 2k \text{ for some } k > 0\}$

$S = \{j : j \text{ is nonnegative and even} \}$

- Universal Set:** all possible elements

$A = \{1, 2, 3, 4, 5\}$



Universal Set: $U = \{1, \dots, 10\}$

Set Operations:

$$A = \{1, 2, 3\} ; B = \{2, 3, 4, 5\}$$

Set Operations:

$$A = \{1, 2, 3\} ; B = \{2, 3, 4, 5\}$$

- **Union:** $A \cup B = \{1, 2, 3, 4, 5\}$

Set Operations:

$$A = \{1, 2, 3\} ; B = \{2, 3, 4, 5\}$$

- **Union:** $A \cup B = \{1, 2, 3, 4, 5\}$
- **Intersection:** $A \cap B = \{2, 3\}$

Set Operations:

$$A = \{1, 2, 3\} ; B = \{2, 3, 4, 5\}$$

- **Union:** $A \cup B = \{1, 2, 3, 4, 5\}$

- **Intersection:** $A \cap B = \{2, 3\}$

- **Difference**

$$A - B = \{1\}$$

$$B - A = \{4, 5\}$$

Mathematical Preliminaries

Set Operations:

$$A = \{1, 2, 3\} ; B = \{2, 3, 4, 5\}$$

- **Union:** $A \cup B = \{1, 2, 3, 4, 5\}$

- **Intersection:** $A \cap B = \{2, 3\}$

- **Difference**

$$A - B = \{1\}$$

$$B - A = \{4, 5\}$$

- **Complement**

Universal set, $U = \{1, \dots, 7\}$

$$A = \{1, 2, 3\} \longrightarrow \bar{A} = \{4, 5, 6, 7\}$$

$$\overline{\{\text{even integers}\}} = \{\text{odd integers}\}$$

DeMorgan's Laws:

- $\overline{A \cup B} = \bar{A} \cap \bar{B}$
- $\overline{A \cap B} = \bar{A} \cup \bar{B}$

Empty, Null Set: ϕ

- $\phi = \{\}$
- $S \cup \phi = S$
- $S \cap \phi = \phi$
- $S - \phi = S$
- $\phi - S = \phi$
- $\bar{\phi} = \text{Universal Set}$

Mathematical Preliminaries

- **Subset:**

- $A = \{1, 2, 3\}; B = \{1, 2, 3, 4, 5\}$
 $A \subseteq B$

- **Disjoint Sets:** a collection of sets where any two distinct sets in the collection have no elements in common.

- $A = \{1, 2, 3\}; B = \{5, 6\}$
 $A \cap B = \phi$

- **Set Cardinality:**

- For finite sets, $A = \{2, 5, 7\}$
 $|A| = 3$

- **Powersets:** A powerset is a set of sets

- $S = \{a, b, c\}$
Powerset of S = the set of all the subsets of $S = 2^S$
 $2^S = \{\phi, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$

- **Observation:**

$$|2^S| = 2^{|S|}$$
$$(8 = 2^3)$$

- **Cartesian Product:**

- $A = \{2, 4\}; B = \{2, 3, 5\}$

- $A \times B = \{(2, 2), (2, 3), (2, 5), (4, 2), (4, 3), (4, 5)\}$

- $|A \times B| = |A||B|$

- Generalizes to more than two sets:

- $A \times B \times \dots \times Z$

• RELATIONS:

- Let A & B be sets. A binary relation “R” from A to B:

$$R = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots\}$$

Where $x_i \in A$ and $y_i \in B$ $R \subseteq A \times B$

- $x_i R y_i$ denotes $(a, b) \in R$
if $R = '>': 2 > 1, 3 > 2, 3 > 1$

• Equivalence Relations

- Reflexive: xRx
- Symmetric: $xRy \implies yRx$
- Transitive: xRy and $yRz \implies xRz$

- **PROOF TECHNIQUES:**

- Proof by induction
- Proof by contradiction

- **Proof by induction:**

- 1 We have statements $P_1, P_2, P_3, \dots$
- 2 If we know:
 - for some b that $P_1, P_2, \dots, P_b$ are true
 - for any $k \geq b$ that $P_1, P_2, \dots, P_k$ imply P_{k+1}
 - Then
Every P_i is true

- **Proof by induction:**

- ① **Inductive basis:** Find $P_1, P_2, P_3, \dots$ which are true.
- ② **Inductive hypothesis:**
 - Let's assume $P_1, P_2, \dots, P_b$ are true for any $k \geq b$
- ③ **Inductive step:** Show that P_{k+1} is true

- **Example:** Proof by induction

Show that $S_n = \sum_{i=0}^n i = \frac{n(n+1)}{2}$

- 1 **Inductive basis:**

$$S_1 = \sum_{i=0}^1 i = \frac{1(1+1)}{2}$$

- 2 **Inductive hypothesis:**

Assuming that $S_n = \sum_{i=0}^n i = \frac{n(n+1)}{2}$ for all $n \geq 1$

- 3 **Inductive step:**

$$S_n = \sum_{i=0}^n i = S_n + (n+1) = \frac{n(n+1)}{2} + (n+1) = \frac{(n+1)(n+2)}{2}$$

- **Proof by Contradiction:**

We want to prove that a statement P is true:

- ① we assume that P is false
- ② then we arrive at an incorrect conclusion
- ③ therefore, statement P must be true

Mathematical Preliminaries

- **Example** Proof by Contradiction:

Theorem: $\sqrt{2}$ is not rational.

Proof:

① Assume by **contradiction** that it is rational

② then $\sqrt{2} = n/m$

n and m have no common factors

We will show that this is impossible

③ $\sqrt{2} = n/m \implies 2m^2 = n^2$

Therefore, n^2 is even $\implies n$ is even ; $n = 2k$

$2m^2 = 4k^2 \implies m^2 = 2k^2 \implies m$ is even; $m = 2p$

Thus, m and n have common factor 2; This is a Contradiction

Table of Contents

- 1 Introduction
- 2 Mathematical Preliminaries
- 3 Formal Language**
- 4 Schedule of Lectures

- **Finite set of alphabets, Σ :**

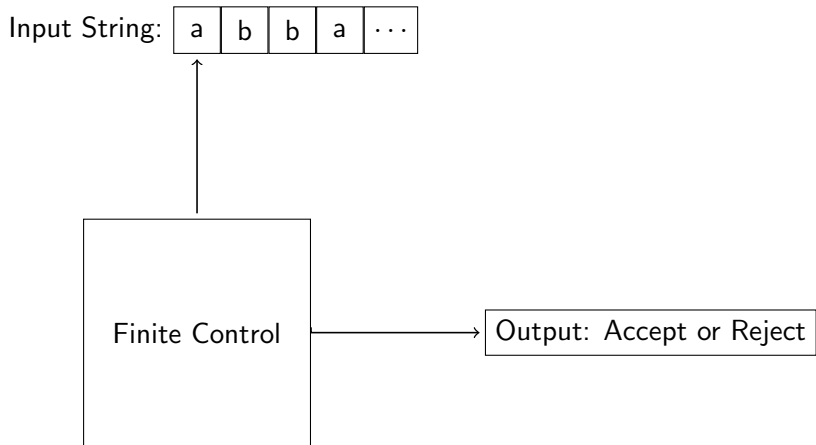
Example: $\{0, 1\}$, $\{a, b, c\}$

- **Language L is a subset of strings on Σ .**

Examples:

- $\{00, 110, 01\}$ is a Finite Language.
- or, $\{\text{Strings starting with } 0\}$ is an infinite Language.
- Σ^* is a special language with all possible strings on Σ

Finite State Machine



Finite State Machine

- Tape, broken into cells
- Tape head.
- Finite control, that is, a program that contains the position of the read head, the current symbol that is being scanned, the current state, etc.
- A string is placed on the tape, read head is positioned at the left end, and the machine reads the string one symbol at a time until all symbols have been read, and then either accepts or rejects the input (to be in the language or not)

Table of Contents

- 1 Introduction
- 2 Mathematical Preliminaries
- 3 Formal Language
- 4 Schedule of Lectures

- **Introduction**

- Computations
- Different models of computation,
- Language recognizer and generator.

- **Regular Languages:**

- Finite Automata – Deterministic and non deterministic,
- Regular expression, regular grammar,
- Equivalence of regular languages,
- Pumping lemma, Myhill-Nerode Theorem,
- Minimization of FSM,
- Properties of the class of Regular languages,
- Decision algorithm for regular sets.

- **Context Free Languages**

• **Recursive and Recursively enumerable Language:**

- Unrestricted grammar,
- Computable function,
- Turing Machines (deterministic and non deterministic),
- Equivalence of deterministic and non deterministic TM,
- Extensions of TM and their simulations,
- Universal TM, Halting problem of TM,
- Decidability, Non-computability, Complexity classes, notion of reductions

• **Recursive and Recursively enumerable Language:**

- Unrestricted grammar,
- Computable function,
- Turing Machines (deterministic and non deterministic),
- Equivalence of deterministic and non deterministic TM,
- Extensions of TM and their simulations,
- Universal TM, Halting problem of TM,
- Decidability, Non-computability, Complexity classes, notion of reductions

References

- Denial I. A. Cohen Introduction to Computer Theory, Second Edition, John Wiley & Sons, 1997.
- J. E. Hopcroft, R. Motwani, & J. D. Ullman Introduction to Automata Theory, Languages, and Computation, Third Edition, Pearson, 2008.